

DL065 推导与实验答案

本答案强调推理步骤和结论边界。性能数字依赖机器负载，复跑不应以逐位相同的秒数为目标；数据、模型结构、训练目标和统计方法才是可核对的条件。

1 三层模型的参数与字节

第一个线性层权重32乘128共4096个，偏置128个，合计4224。第二层128乘128加128，合计16512。输出层128乘2加2，合计258。三个数相加得到20994。

FP32每个元素4字节，因此参数83976字节。梯度存在且同dtype同形时也是83976字节。Adam的两个主要状态各一份参数大小，共167952字节；本课有6个参数张量，每个state还存一个FP32步数标量，再增加24字节，实际优化器张量状态167976字节。总逻辑载荷335928字节。

这个合计不是总训练内存。它没有保存激活、输入、输出、临时工作区、分配器缓存和Python对象。若存在共享存储，仅按numel相加还会重复计数；本课网络没有这类共享。

1000到100层：参数100000加100等于100100个；参数400400字节，参数加梯度800800字节，加两个主要Adam状态为1601600字节。除以1048576得到约1.5274 MiB。若进一步计入两个参数张量的步数标量，再加8字节；题目要求主要载荷时，应明确是否包括这8字节。

2 延迟与吞吐不等价

批量128、20步共0.08秒，每步0.004秒，即4毫秒。共处理2560个样本，吞吐32000样本/秒。

批量256、20步共0.12秒，每步6毫秒，共处理5120个样本，吞吐约42666.7样本/秒。因此单步延迟变大50%，处理吞吐提高约33.3%。它可能适合吞吐导向训练，却不自动适合低延迟推理，也不自动让模型更早达到同一验证误差。

讲义自测中的64样本、10步共0.02秒，结果为2毫秒/step和32000样本/秒。若把0.02秒错当成每步时间，会把吞吐低估10倍；检查计时块的分母非常重要。

3 异步计时的两个边界

GPU函数返回可能只说明工作已进入队列，尚未执行完成。末尾不同步会把真正计算排除在计时之外；开头不同步可能让上一个区间遗留的工作计入当前区间。正确墙钟测量在开始前与结束后都建立完成边界。

CUDA事件量的是相关设备流上的时间，墙钟量的是主机观察到的区间，两者问题不同。不同步的主机提交延迟也可以是合法指标，但必须被命名为提交延迟，不能拿去宣称完整GPU训练step耗时。

每个算子后都同步虽然容易解释，却会破坏原本可重叠的流程。稳态整体吞吐应按完整工作块测量；分析单个算子则独立使用分析器。不要为了得到方便的数字，改变了原本想评估的执行方式。

4 Adam为什么首次更新前可能没有状态

优化器可以先保存参数引用与超参数，等真正看到梯度才为对应参数分配动量状态。这样，没有参与训练的参数不必提前承担全部状态成本。memory_ledger在构造后看到状态为0，第一步后看到状态出现，与这个实现策略一致。

它不表示第一次更新“凭空增加了参数”。参数一直存在，增加的是优化器用于未来更新的历史信息。若用SGD无动量，状态会不同；若用混合精度或分片优化器，也要按实际state检查，不能机械套用16P。

5 为什么逐行与向量化在这里可以比较

两种实现对每个样本使用相同确定性网络，最终都把所有logits组成同一个批，计算同一个平均交叉熵，只做一次更新。没有跨样本的BatchNorm，也没有随机Dropout，因此数学目标一致。

浮点运算归约顺序可能不同，故使用带容差的梯度和参数测试。这个测试比仅比较一个loss更强：两个错误有时能碰巧给出相近loss，却产生不同更新方向。测试也比较更新后参数，确保没有漏掉或重复执行optimizer.step。

如果每行都更新一次，第2行使用的已经是被第1行修改的参数。原本整批在共同参数点上求梯度的目标被改成序列更新；这不是相同目标的单纯实现优化。Adam的时间步与动量也随之变化。

6 峰值与事件计数

假设A在时刻1到3存活占8 MiB，B在时刻4到6存活占12 MiB。它们最大单独载荷相加为20 MiB，但从不同时存活，理想活动载荷峰值只有12 MiB。若有1到6一直存活的参数4 MiB，峰值为16 MiB。

反过来，外部分配器可能保留释放后的块，保留内存仍可能高于活动载荷。一个view和原张量共享存储，逻辑元素计数可能重复。profiler中归属某事件的分配与释放不是一张完整峰值照片；负的self memory不等于“这个算子使用了负内存”。

本课CPU数据没有给出RSS峰值，也没有CUDA峰值。null是未测，0是一个数值测量；两者含义不能混淆。

7 Amdahl定律

把原总时间归一化为1。60%不可变部分仍花0.6，40%可变部分加速4倍后花0.1，所以新总时间0.7，整体加速比为1除以0.7，约1.4286。

即使可变部分时间趋于0，整体仍需0.6，最大加速比为1.6667。这个上限依赖“剩余部分不变、无新增开销、可加时间边界”等假设。优化可能改变缓存、数据搬运和重叠，因此实际还要验证。

若只公布“某内核4倍”，读者无法知道它原本占总时间多少。要支持整体结论，必须给出该比例以及完整训练或端到端实测。

8 一份不公平比较及修正

错误版本：把A的从文件加载到保存checkpoint时间，与B仅前向计算时间比较；同时把B的批量提高4倍并减少epoch，然后说“B训练快了10倍”。这里计时边界、处理样本量和优化预算都变了。

修正之一是实现级比较：固定模型、参数初值、输入张量、批量、loss归约、dtype和更新次数；两边都计清梯度到更新结束，独立预热，保存多个测量块，并检验梯度。这个协议回答计算实现是否更快。

修正之二是训练级比较：预先固定验证指标门槛、验证频率、数据划分、超参数搜索预算、种子与最大训练预算；两边都包含必要数据管道并报告达到门槛的用时。它回答在指定任务与预算下谁更早达到某一目标，不能与实现级比较混用。

9 达到同样质量的实验方案

例如在当前合成分类任务上，把验证准确率至少0.90设为门槛，用独立验证集判断，每个epoch验证一次。预先选择种子11、23、37、最大30个epoch，测试集只在最终报告时使用。每个配置从全新进程开始，计时包含训练与必要验证，保存首次达到门槛的时刻与后续曲线。

如果某次始终未达门槛，记录“30个epoch预算内未达到”，而非删除它或把最大用时当成成功时间。比较成功率以及成功运行的用时，同时披露删失。若门槛选在看到结果之后，也必须说明这是探索性分析并需要新一轮验证。

这个方案要新增验证集，而不能把目前讲义报告的测试集既当门槛调优集又当最终独立测试。数据角色是证据的一部分。

10 实测复核与可以说出的结论

保存checkpoint重新加载后，测试准确率0.92578125。普通与-O各执行11项测试通过。完整计时值与分析器条目保存在results.json和profiler.txt；这些数值不要求跨机器相等，准确率与配置则能在本课固定CPU流程中复核。

可以说：这组受控CPU实验支持去掉逐行Python模型调用提高驻留数据吞吐，且目标通过梯度/更新测试保持一致。不能说：它证明任意GPU上的加速倍数，或证明只需335928字节就能训练该模型，或证明批量越大收敛越快。

下一步若想研究GPU峰值，就需要真实CUDA运行、独立进程测量与明确的对象生存期。若想研究用户等待时间，就需要把读取、传输、验证和保存纳入边界。提出新问题时，先改变测量协议，再改变结论。